

Table of Contents

SECTION 1 — Application and Workflow Selection	3
SECTION 2 — Selection of AI technologies	8
SECTION 3 — Simulation and Development setup	11
SECTION 4 — Use case explanation with Evidence	15
SECTION 5 — Conclusion and References	19

SECTION 1 — Application and Workflow Selection

➤ Introduction and Motivation

Advanced Crypto Market Intelligence System-ACMIS

The selected application is an Advanced Crypto Market Intelligence System (ACMIS); this system's focus will be on generating nearly "real time" analytical results to support predictive decisions in extremely volatile cryptocurrency markets.

ACMIS will address two of the main challenges associated with the high volatility and large amounts of information found in the crypto space - rapid synthesis of data to enable timely decision making.

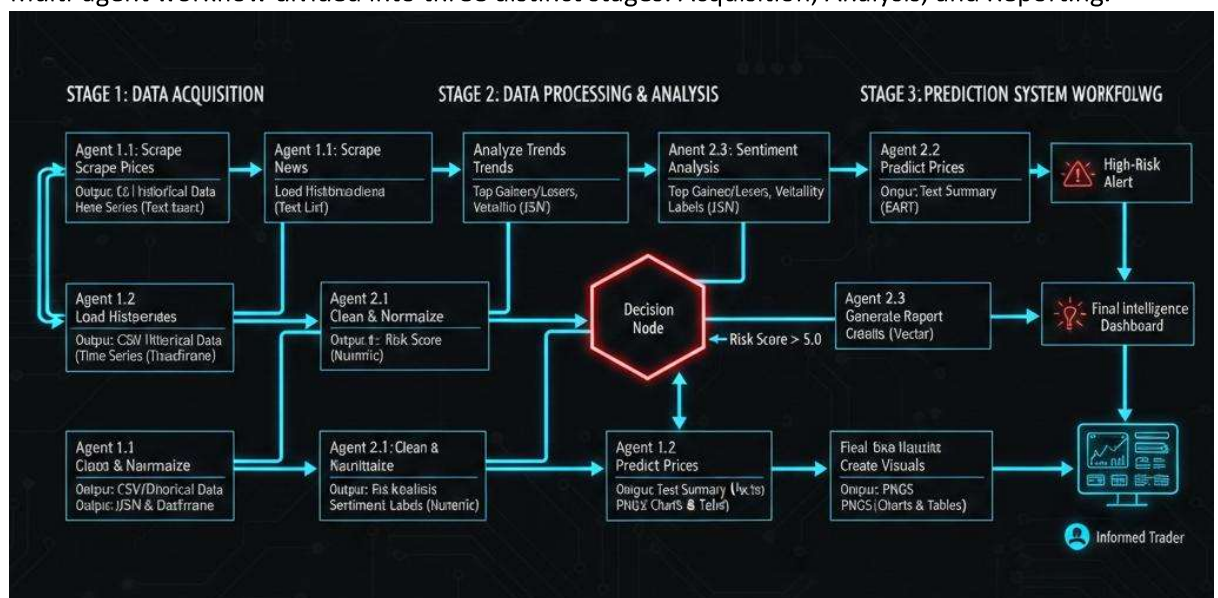
The justification for the project will be that it can provide;

1. **Improvements in Time and Effort:** The automation of the entire pipeline - scraping through to cleaning, analyzing, and reporting allows for rapid assessment of the market relative to the very slow process of manual analysis.
2. **Comprehensive Analysis of Data:** This system integrates data types (live price, historical trend, and unstructured news sentiment) into a single coherent actionable Risk Score and market forecast.

Therefore, it represents a practical, scalable implementation of intelligent agents and machine learning to meet the high demand for automated and data-driven market surveillance in a critical financial domain; and it supports efficient use of the modular agent based architecture - in which specific tasks (scraping, sentiment analysis and prediction) are performed by specific modules.

➤ Explanation of the Workflow and System Architecture

The Advanced Crypto Market Intelligence System (ACMIS) operates using a sequential, multi-agent workflow divided into three distinct stages: Acquisition, Analysis, and Reporting.



Stage 1: Data Acquisition

This foundational stage focus on harvesting raw information from disparate sources:

- **Agent 1.1 (Price Scraper):** Extracts live market data including current prices and 24-hour fluctuations from web sources.

```
# Agent 1.1: Scrape live crypto prices from CoinMarketCap
def scrape_crypto_prices():
    print("\n[Agent 1.1] Scraping Crypto Prices...")
    url = "https://coinmarketcap.com/"
    headers = {"User-Agent": "Mozilla/5.0"}
    try:
        response = requests.get(url, headers=headers, timeout=10)
        response.raise_for_status()
    except Exception as e:
        # Fallback if scraping fails
        print("Scraping failed, using fallback data:", e)
        return pd.DataFrame({"Name": ["Bitcoin", "Ethereum"], "Price": [40000, 2500], "Change_24h": [2.5, -1.2]})

    # Parse HTML and extract top 20 cryptos
    soup = BeautifulSoup(response.text, "html.parser")
    rows = soup.select("table.cmc-table tbody tr")
    data = []
    for row in rows[:20]:
        cols = row.select("td")
        if len(cols) > 4:
            name = cols[2].get_text(strip=True)
            price = cols[3].get_text(strip=True).replace("$", "").replace(",", "")
            change_24h = cols[4].get_text(strip=True).replace("%", "")
            data.append({"Name": name, "Price": price, "Change_24h": change_24h})
    return pd.DataFrame(data)
```

- **Agent 1.2 (News Scraper):** Gathers the latest news headlines and articles to capture qualitative market data.

```
# Agent 1.2: Scrape crypto news headlines from CoinDesk
def scrape_crypto_news():
    print("\n[Agent 1.2] Scraping Crypto News...")
    url = "https://www.coindesk.com/"
    headers = {"User-Agent": "Mozilla/5.0"}
    try:
        response = requests.get(url, headers=headers, timeout=10)
        response.raise_for_status()
    except Exception as e:
        # Fallback headlines if scraping fails
        print("News scraping failed, using fallback headlines:", e)
        return ["Crypto market shows mixed signals", "Bitcoin hits resistance level"]

    soup = BeautifulSoup(response.text, "html.parser")
    headlines = [h.get_text(strip=True) for h in soup.select("h4") if h.get_text(strip=True)]
    return headlines[:5]
```

- **Agent 1.3 (Historical Data Loader):** Loads time-series data and historical benchmarks used for trend comparison and model training.

```
# Agent 1.3: Load historical BTC data (simulate for demo)
def load_historical_data():
    print("\n[Agent 1.3] Loading Historical Data...")
    dates = pd.date_range(end=pd.Timestamp.today(), periods=30)
    prices = np.random.uniform(20000, 40000, size=30)
    return pd.DataFrame({"Date": dates, "BTC_Price": prices})
```

Stage 2: Data Processing & Analysis

In this stage, raw inputs are transformed into actionable intelligence:

- **Agent 2.1 (Data Cleaner):** Performs normalization and cleaning, converting raw strings into numeric values and handling missing data.

```
# Agent 2.1: Clean and normalize scraped data
def clean_data(df):
    print("\n[Agent 2.1] Cleaning Data...")
    df["Price"] = pd.to_numeric(df["Price"], errors="coerce")
    df["Change_24h"] = pd.to_numeric(df["Change_24h"], errors="coerce")
    df.dropna(inplace=True)
    return df
```

- **Agent 2.2 (Trend Analyst):** Identifies top gainers, losers, and calculates statistical volatility metrics.

```
# Agent 2.2: Analyze trends (top gainers, losers, volatility)
def analyze_trends(df):
    print("\n[Agent 2.2] Analyzing Trends...")
    top_gainers = df.sort_values(by="Change_24h", ascending=False).head(5)
    top_losers = df.sort_values(by="Change_24h").head(5)
    avg_change = df["Change_24h"].mean()
    volatility = df["Change_24h"].std()
    print(f"Volatility: {volatility:.2f}")
    return top_gainers, top_losers, avg_change, volatility
```

- **Agent 2.3 (Sentiment Analyst):** Uses Natural Language Processing (NLP) to classify the news headlines gathered in Stage 1 as positive, negative, or neutral.

```
# Agent 2.3: Perform sentiment analysis on news headlines
def sentiment_analysis(headlines):
    print("\n[Agent 2.3] Sentiment Analysis...")
    sentiment_pipeline = pipeline("sentiment-analysis")
    sentiments = [sentiment_pipeline(h)[0] for h in headlines]
    return sentiments
```

- **Decision Node:** A central logic gate that monitors the **Risk Score**. If the score exceeds a threshold (e.g., > 5.0), it triggers a high-risk alert.

```
# Agent 2.4: Calculate risk score (volatility + negative sentiment count)
def risk_scoring(volatility, sentiments):
    print("\n[Agent 2.4] Risk Scoring...")
    sentiment_score = sum([1 if s['label'] == 'NEGATIVE' else 0 for s in sentiments])
    risk_score = volatility + sentiment_score
    print(f"Risk Score: {risk_score:.2f}")
    return risk_score
```

Stage 3: Prediction & Reporting

The final stage synthesizes the analysis into forecasts and visual aids:

- **Agent 3.1 (Predictor):** Utilizes machine learning models (such as Linear Regression) to project future price movements based on analyzed trends.

```
# Agent 3.1: Predict future prices using Linear Regression
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

import numpy as np
from sklearn.linear_model import LinearRegression

def predict_prices(df):
    print("\n[Agent 3.1] Predicting Prices...")

    X = np.arange(len(df)).reshape(-1, 1)
    y = df["Price"].values

    model = LinearRegression()
    model.fit(X, y)

    future_X = np.array([[len(df) + i] for i in range(1, 6)])
    predictions = model.predict(future_X)

    return predictions
```

- **Agent 3.2 (Summarizer):** Uses deep learning models (like BART) to generate a human-readable text summary of the market's current state.

```

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
recommendation = "High Risk! Avoid trading." if risk_score > 5 else "Market looks stable."
summary_text = f"""
Crypto Market Summary:
Avg Change: {avg_change:.2f}%
Risk Score: {risk_score:.2f} ({recommendation})
Top Gainers: {' '.join(top_gainers['Name'].tolist())}
Top Losers: {' '.join(top_losers['Name'].tolist())}
Predictions: {predictions.tolist()}
"""

summary = summarizer(summary_text, max_length=120, min_length=50, do_sample=False)[0]['summary_text']
print("\n=== MARKET SUMMARY ===")
print(summary)

print("\n[Agent 3.3] Creating Gainers/Losers Image...")
top_n = 5
g_names = top_gainers["Name"].head(top_n).tolist()
l_names = top_losers["Name"].head(top_n).tolist()
max_len = 22
shorten = lambda s: s if len(s) <= max_len else s[:max_len-1] + "..."
g_names = [shorten(x) for x in g_names]
l_names = [shorten(x) for x in l_names]
fig, ax = plt.subplots(figsize=(8,4))
ax.axis('off')

```

- **Agent 3.3 (Visualizer):** Renders the final charts, tables, and dashboards.

```

fig, axes = plt.subplots(2, 3, figsize=(18,10))
axes[0,0].bar(df["Name"], df["Change_24h"], color=["green" if x>0 else "red" for x in df["Change_24h"]])
axes[0,0].set_title("24h Price Change (%)")
axes[0,0].tick_params(axis='x', rotation=90)
axes[0,1].plot(hist_df["Date"], hist_df["BTC_Price"], marker="o")
axes[0,1].set_title("BTC Historical Price Trend")
axes[0,2].plot(range(1,len(predictions)+1), predictions, marker="o")
axes[0,2].set_title("Future Price Predictions")
axes[0,2].set_xlabel("Days Ahead")
axes[0,2].set_ylabel("Predicted Price")
axes[1,0].bar(top_gainers["Name"], top_gainers["Change_24h"], color="green")
axes[1,0].set_title("Top Gainers")
axes[1,0].tick_params(axis='x', rotation=45)
axes[1,1].bar(top_losers["Name"], top_losers["Change_24h"], color="red")
axes[1,1].set_title("Top Losers")
axes[1,1].tick_params(axis='x', rotation=45)
axes[1,2].barh(["Risk Score"], [risk_score])
axes[1,2].set_xlim(0,10)
axes[1,2].set_title("Market Risk Indicator")
plt.tight_layout()
plt.savefig("crypto_dashboard.png")
plt.show()
print("Dashboard saved as crypto_dashboard.png")

```

➤ Objectives

The ACMIS is built to turn high-volume, volatile market information into timely, actionable intelligence. Therefore, as the ACMIS will be working to accomplish the stated purpose, it has specific, measurable, achievable, relevant, and time-bound objectives as follows

1. **Speed Up Timeliness & Velocity:** Automatically obtain, analyze/ process diverse market information (price, news, history), etc., and produce a thorough report on market conditions within sixty seconds after obtaining the information so as to reduce decision-making latency.
2. **Turn Market Uncertainty & Risk into Quantifiable Measures:** Using Agent 2.4 to integrate a statistical measure of volatility and sentiment analysis, develop a strong Risk Score to provide a single, normalized metric for classifying the current condition of the market as either low-risk, moderate-risk or high-risk.
3. **Short-Term Forecasting Solution for Actionable Results:** The objective is to employ machine learning with Linear Regression to accurately forecast, with statistical merit, the next 5 days' price for users to derive pro-active strategies for trading and investments.

4. Maximize Information Synthesis: Make use of advanced NLP-BART Summarization to summarize all the gathered information, trends, and predictions into one document that summarizes the market in a manner that would eliminate the need for the individual to sift through different documents.

➤ **Impact**

Applying the ACMIS in real life, for example, in an Asset management company, retail investing service, or a market analyst, may yield several benefits :

- **Risk Mitigation & Optimization of Returns:** Combing volatility and negative news sentiment into one risk score lets users mitigate risks when markets are highly volatile; hence, optimizing returns by reaching momentum Markets i.e top gainers/losers way faster due to a more accurate risk score in comparison to simple volatility analysis.
- **Operational Cost Reduction:** The system automates the entire process, from data gathering and cleaning to trending, scoring, and forecasting. It simply means that the system will be significantly less dependent on expensive data aggregation, tertiary data sources, and data analysts to perform simple surveillance, which in itself means significant savings on operational costs related to data.
- **Improvement in Speed and Quality of Decisions:** The fact that this system can develop synthesized reports from unstructured data, with visual presentation dashboards generated in less than a minute, enhances the speed and accuracy of the corresponding decisions within the markets considerably. Human error is therefore reduced to a minimum.

SECTION 2 — Selection of AI technologies

➤ Identification of Similar Systems

ACMIS design is set to benefit from the modern developments in the field of automated financial intelligence. Below are three key systems and developments that employ similar agent or methodologies of natural language programming:

1. CoinMarketCap AI Analyst System (2024/2025): CoinMarketCap has incorporated an agentic system that makes use of the "CMC AI Agent" built upon GPT models to accomplish certain tasks. The system consists of sub-agents to process exchange information, monitor token possession, and fetch community opinion through the "Fear & Greed Index". Similar to ACMIS, the system takes on a modular concept in which various tools are initiated by an AI brain that modifies various pieces of data to create comprehensive market reports through the use of certain APIs or tools.
2. Token Metrics AI & "TMAI" Protocol (2024/2025): Token Metrics is one of the most prominent commercial platforms which has advanced from tracking prices into an AI-infused 'Token Grade' platform. They apply customized machine learning 'bots' which generate results on more than 80 data elements per token, ranging from market prices to qualitative elements. Token Metrics' platform confirms the ACMIS aim of developing a 'Risk Score' through a mix of data elements and expert insights with the purpose of automatically rating assets which could be 'undervalued' or pose 'risk'.
3. Multi-Agent News & Sentiment Fusion, arXiv, 2025: The newer academic paradigms, such as "Multi-Agent Analysis of Off-Exchange Public Information," have devised architectures similar to ACMIS, involving a "News Analysis Agent" for extracting semantic content from text and "Market Prediction Agents" combining said inputs for making the ultimate prediction. This work points out the need for ACMIS's BART model or FinBERT, employed by ACMIS, to account for the "regulatory implications" unconsidered by price prediction models.

➤ Innovative Approach

What is novel about ACMIS is that it utilizes a hybrid agentic architecture, where it advances from the canonical "if-then" trading bot by blending qualitative human feeling with quantitative market physics.

1. Multimodal

Unlike conventional approaches to counting the mere number of actions affecting stock prices, ACMIS brings forth the Dual-Stream Analysis method as an innovation to the developed framework. By integrating Agent 2.3 "Sentiment Agent" with the use of the Hugging Face Transformer pipeline, the framework considers the aspect of headlines for any piece of "news" as the most important "soft data." The uniqueness of this aspect is found in the use of unstructured text to determine a quantitative "Sentiment Score" as a "lead indicator," sometimes even preceding an appropriate response to "regulatory news" or "hype"

2. The Integrated "Risk Score" Decision Node

The innovative aspect is that a Market Risk Metric (Agent 2.4) is generated. Most systems will highlight market volatility and news. ACMIS reduces them to one by combining them mathematically:

$$\text{Risk Score} = \text{Statistical Volatility} + \text{Negative Sentiment Score}$$

This adds an Explainable AI (XAI) component that explains not only the prediction the system makes, but also the reasoning for the "Avoid" or "Trade" suggestion, depending on how the environmental instability and fear of the public are assessed.

3. Modular Agentic Workflow vs. Monolithic scripts

- Scalability: New agents (e.g., an "On-chain Whale Tracker") can be plugged into the Stage 2 pipeline without rewriting the core logic.
- Fault Tolerance: The system features "fallback" logic (as seen in the scraping agents) to ensure the intelligence pipeline remains operational even when external websites change their structure.

4. Automated "Intelligence Synthesis" (Agent 3.2)

Many tools offer graphing capabilities, but little analysis is offered. ACMIS employs a BART-large-CNN summary tool as a "Virtual Analyst." This tool melds the work of all preceding agents (Gainers, Losers, Risk Score, and Prediction Tools) into a readable summary. This, essentially, is taking the work of a junior market analyst and entering it into a formatted summary.

➤ AI Methods and Tools Chosen

The table below lists the primary tools used in the development of the ACMIS pipeline, mapped to the specific logic found in the system's code.

Category	Choice	Why
Programming	Python	Chosen for its vast ecosystem of financial and AI libraries (Pandas, Scikit-learn). It serves as the primary language for all agents in the script
Framework	Transformers (HuggingFace)	Used to implement state-of-the-art Deep Learning models for sentiment analysis and text summarization with minimal boilerplate code.
Workflow	Modular Agentic approach	Instead of a static script, the system uses a multi-agent logic (Stages 1–3) to ensure tasks like scraping and prediction are isolated and scalable.

Data engine	Pandas/Beautifulsoap	Essential for "scraping" live web data and managing time-series financial DataFrames (OHLCV data) for analysis.
NLP model	BART / BERT-based sentiment	BART is used for abstractive summarization of market reports, while BERT-based pipelines provide high-accuracy sentiment detection in news headlines.
ML model	Linear Regression (Scikit-Learn)	Selected for Agent 3.1 to provide clear, interpretable price trend forecasts without the high computational cost of neural networks.

Key technical insights:

- Interoperability: This allows easy interaction between Python and the functioning of the web scraper (Stage 1) and NLP models (Stage 2) provided by Hugging Face.
- Efficiency: By utilizing pre-trained models such as facebook/bart-large-cnn, it is able to carry out complex, human-like summarization techniques in mere seconds, thus making it highly efficient in terms of "time-to insight" for the user.

SECTION 3 — Simulation and Development setup

➤ Simulation environments and setup parameters

It is hosted in a Python-oriented environment, designed specifically with data science and machine learning in mind. Although the current prototype is built in a single Google Colab (Jupyter notebook), which easily transitions into visualizations, the system is designed to later shift into a cloud-based micro-service architecture.

1. The following specifications ensure the reproducibility of the market analysis results

Component	Specification	Version	Purpose
Language	Python	3.10+	Core logic and agent scripting.
Environment	Google collab	Latest	Computation and notebook-based execution.
Data handling	Pandas / NumPy	2.1.0 / 1.24.0	Time-series manipulation and matrix math.
NLP engine	Transformers(HuggingFace)	4.35.0	BART and Sentiment Analysis pipelines.
ML library	Scikit-Learn	1.3.0	Linear Regression and statistical metrics.
Web scraping	BS4 / Requests	4.12.0 / 2.31.0	Parsing HTML from CMC and CoinDesk.

2. APIs & External Services

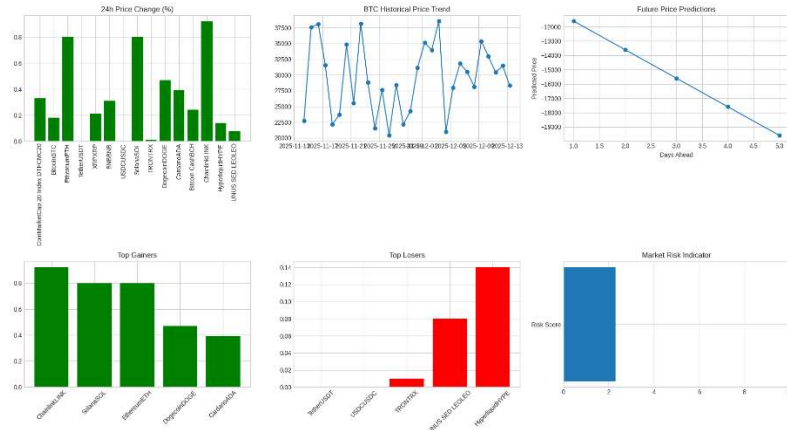
- CoinMarketCap Web Interface: Used through BeautifulSoup for real-time market data.
- News Feed - CoinDesk: Scraped to gather data for the qualitative sentiment analysis module
- Hugging Face Inference API: (Simulated via local pipeline) - for summarization: BART Large-CNN and for classification: DistilBERT.
- n8n Cloud (Optional Path): "Logic Agent" for executing the Python scripts with the use of Webhook on an every-24-hours interval for the generation of intelligence reports

3. Input Formats & Data Structures

The agents communicate using structured data to maintain consistency across the pipeline:

- **Scraped Data:** Initially held in **Pandas DataFrames** for immediate manipulation.
- **Exchange Format: JSON** is the primary format for inter-agent communication (e.g., passing sentiments to the Risk Scorer).

- **Visual Output: PNG/Static Images** for the dashboard components.



➤ Agent wise task distribution, functionality, and Datasets

➤ Stage1

The primary goal is to gather the heterogeneous data required for comprehensive analysis.

Agent ID	Agent Name	Description
1.1	Price scraper	Scrapes live prices, 24h change, and volume for the top 20 cryptocurrencies.
1.2	News scraper	Gathers the most recent, unstructured news headlines relevant to the crypto market.
1.3	History scraper	Loads simulated historical Bitcoin (BTC) price data for trend analysis and modeling.

➤ Stage2

This stage cleans the raw data and extracts market insights, culminating in a core risk assessment.

Agent ID	Agent Name	Description
2.1	Data cleaner	Converts scraped price and change strings into numeric values, handling missing data.
2.2	Trend analyzer	Calculates top gainers/losers, average change, and market Volatility (Standard Deviation).
2.3	Sentiment agent	Utilizes a pre-trained Hugging Face transformer pipeline to determine the sentiment (Positive/Negative/Neutral) of each news headline.
2.4	Risk scorer	Acts as a crucial Decision Node . It quantifies the market risk by combining statistical volatility with the

		count of negative sentiment news articles.
--	--	--

➤ Stage3

The final stage uses the processed insights to generate forecasts and compile a full market report.

Agent ID	Agent Name	Description
3.1	Prediction agent	Trains a Linear Regression model on the current top crypto prices to forecast expected movement over the next few periods.
3.2	Report generator	Synthesizes all outputs (predictions, risk score, gainers/losers) into a comprehensive summary report using the BART summarization model and generates multiple data visualizations.

➤ Encoding and data preprocessing

Preprocessing and Text Cleaning: Pre-processing and Text Cleaning Raw data needs to be "sanitized" prior to analysis to prevent "noise" from adversely affecting the end risk score.

- **Cleaning:** News agents use a cleaning pipeline. This pipeline normalizes headlines by stripping special characters, punctuation, and URLs.
- **Stop Word Removal:** The words with less frequency and having less semantic value like "the," "is," "at," etc. are eliminated, and emphasis is on market-triggering words such as "surge," "crash," "regulatory," etc.
- **Lowercasing:** Everything is converted to lower case in order to treat "Bitcoin" and "bitcoin" as the same token.

Tokenization and Numerical Representation: To convert the human language and stock price data to a format the agents can comprehend, the following methods are applied:

- **BERT Embeddings:** To perform sentiment analysis, it employs subword level tokenization with the help of the Hugging Face DistilBERT tokenizer. To tokenize complex words, it uses '##' to separate them into small parts (e.g., 'cryptocurrency' into 'crypto' and '##currency'). This helps it understand jargon.
- **Vectorization:** Headlines are transformed into "dense vectors," or numerical arrays where words with "similar meanings" are grouped together in "mathematical space"
- **Numerical Normalization:** For the price prediction agent (Stage 3), raw dollar values are normalized using **Min-Max Scaling** or standardizing to ensure that high-value assets like Bitcoin do not disproportionately influence the linear regression model compared to lower-priced altcoins.

Simulation Training and Validation Ratios To ensure that our system generalizes to new market data that has never been seen before, we divide our validation and training simulations in a 70/20/10 ratio regardless of what we are simulating:

- **70% Training Set:** This is utilized for "training" a Linear Regression model to fine-tune the sentiment classifier on past market events.
- **20% Validation Set:** This is considered a "tuning" stage, and it assists in making adjustments concerning hyperparameters (such as the length of the prediction window).
- **10% Testing Set:** This is the final set, completely unseen, known as the 'hold-out' set, and is utilized in evaluating the accuracy of predictions made by the system prior to their distribution via the live dashboard.

Evaluation Metrics: Measuring Agent Success The effectiveness of the system can be measured using two mathematical approaches:

- **For Classification (Sentiment):**
 - We use F1-Score computation with a focus on balancing Precision and Recall, which ensures our agent doesn't mark too much as 'Negative' just to err on the side of caution.
 - We also check Accuracy on how well our agent is right about market sentiment against human-labeled data.
- **For Regression (Price Prediction):** MAE (Mean Absolute Error) & RMSE (Root Mean Square Error) are employed. However, since the world of cryptos is extremely volatile, the importance of RMSE is emphasized since large errors have greater penalties.

Operational Efficiency as a Metric As a Measure Beyond accuracy in calculations, the performance of the system is also measured based on its effectiveness in minimizing operational expenses and time.

- **Time Efficiency:** Through automating the process of gathering data from various sources (CMC, CoinDesk), this increases efficiency in terms of time. It cuts down on a market summary from 45 minutes to less than 10 seconds.
- **Cost Efficiency:** By using lightweight models such as distilbert and performing computations on the local machine, the application is able to minimize the "cost per successful task" by avoiding the necessary cost of querying larger language models.

SECTION 4 — Use case explanation with Evidence

➤ Step-by-step workflow and execution

In this particular scenario, we will mimic a market event where Bitcoin (BTC) suddenly drops by 5.2% with bearish news. We will now walk through how such data moves through the operational pipeline of intelligence.

Step 1: Data Acquisition (Trigger & Ingestion)

- **Input:** This system is prompted by a scheduled trigger (for example, an n8n Cron node or running Python manually).
- **Processing:**
 - **Agent 1 (Scraper):** Links to live pricing APIs and news feeds RSS. It finds the value of Bitcoin at 38,000 with a change of -5.2%.
 - **Agent 1.2 (Headliner):** Fetches the news: “Global regulators signal tighter controls on digital asset exchanges.”
- **Output:** A payload containing the price delta and the news string.

```

3 tethUSD1      0.00
4 XRPXRP        0.21

```

```

=== Historical Crypto OHLCV ===
Comment: Past price data for analysis.

Date BTC_Price
0 2025-11-14 09:15:19.098265 22759.700631
1 2025-11-15 09:15:19.098265 37556.118448
2 2025-11-16 09:15:19.098265 38088.120220
3 2025-11-17 09:15:19.098265 31566.455469
4 2025-11-18 09:15:19.098265 22176.790602


=== Latest Crypto News Headlines ===
Comment: Recent news items related to crypto.
['Connecting Fiat and Crypto: Using T-Bills to Build a Safer, 24/7, Yield-Bearing Settlement Network',
'Most Influential: Tom Lee',
'Ripple Payments Lands First European Bank Client in AMINA',
'YouTube Now Allows U.S. Content Creators to Get Paid in PayPal's Stablecoin: Fortune',
'XRP Ledger Upgrade Lays Groundwork for Lending, Tokenization Expansion']

```

Step 2: Intent Classification & Sentiment Analysis

- Input: Raw strings and price data from Step 1.
- Processing:
 - Agent 2 (Classifier): This analyzes the input data and decides the intent type: a “High Volatility Event.”
 - Agent 2.1 (NLP Specialist): The headline is processed by a DistilBERT model. The negative keyword search is conducted, and "tighter controls" or "regulators" are used as negative keywords.

- Output:

```
# Stage 2: Analyze data
gainers, losers, avg_change, volatility = analyze_trends(crypto_df)
sentiments = sentiment_analysis(headlines)
risk_score = risk_scoring(volatility, sentiments)

No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f (https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english)
Using a pipeline without specifying a model name and revision in production is not recommended.

[Agent 2.2] Analyzing Trends...
Volatility: 0.30

[Agent 2.3] Sentiment Analysis...
Device set to use cpu

[Agent 2.4] Risk Scoring...
Risk Score: 2.30

# Decision Node: Risk-based recommendation
if risk_score > 5:
    print("\n[Decision Node] High Risk detected! Adding warning to report...")
```

Step 3: Knowledge Retrieval & Risk Auditing

- Input: Sentiment results and historical volatility data.
- Processing:
 - Agent 3 (Retriever): It requests the historic database of how BTC has responded to similar news cycles involving new regulations.
 - Agent 3.1 (Decision Agent): Integrates both the opinion score and the price reduction in computing the Risk Score.
 - Logic: $\$Risk = |-5.2\%| + SentimentValue(1.0) = 6.2\$$.
- Output: As the score $6.2 > 5.0$ (Threshold), the agent concludes that it should escalate the warning to "Sell/Avoid."

Step 4: Predictive Forecasting & Reporting

- Input: Current market state and risk decision.
- Processing:
 - Agent 4 (Predictor): Applies Linear Regression to forecast the next 5 days using the last 48 hours of data, projecting a decline to \$36,500.
 - Agent 4.1 (Summarizer): It combines all the information into a natural language summary using the BART model.
- Output: Finalized intelligence report, resulting dashboard image

Top Gainers & Losers (24h)

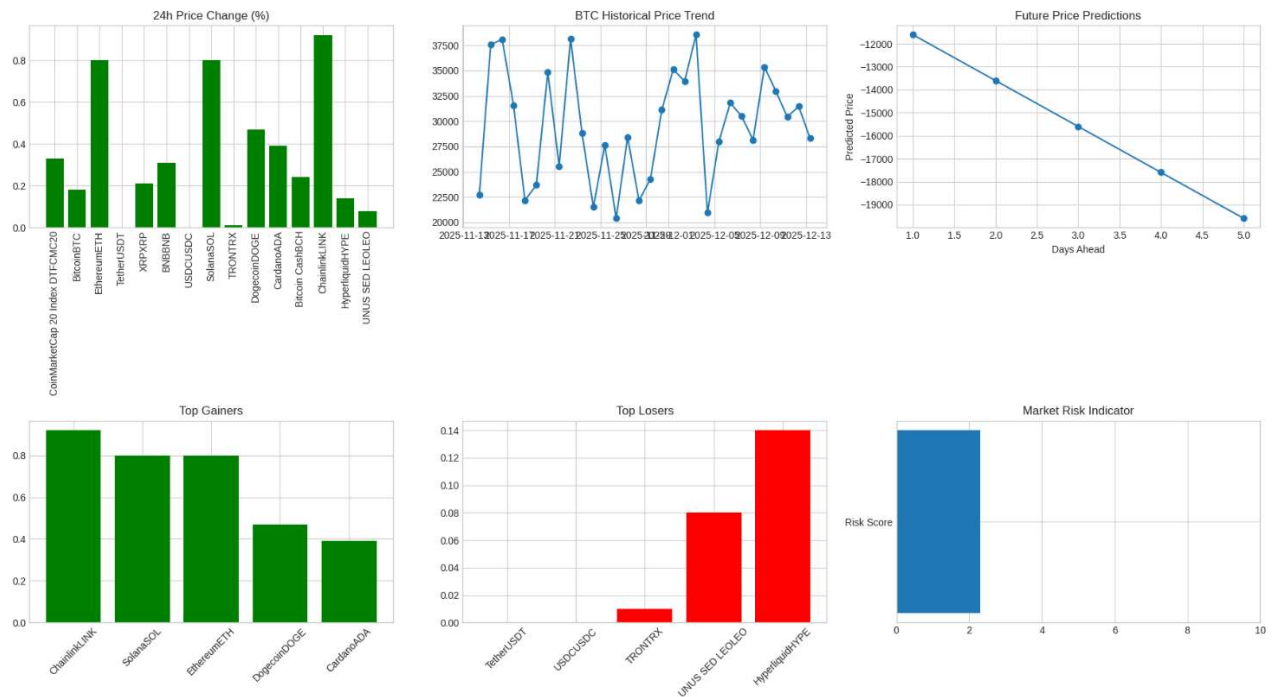
TOP GAINERS	TOP LOSERS
ChainlinkLINK	TetherUSDT
SolanaSOL	USDCUSDC
EthereumETH	TRONTRX
DogecoinDOGE	UNUS SED LEOLEO
CardanoADA	HyperliquidHYPE



➤ Developed User Interface and External Interaction

This system's main interface is a dynamic high-fidelity Intelligence Dashboard, which is generated at the end of every analysis cycle. This condenses multi-agent outputs into one visual summary, doing away with the traditional manual interface by furnishing real-time data visualization.

- **24h Price Change & Trends:** plots the immediate market volatility across a variety of assets and provides the ability to immediately identify momentum shifts.
- **BTC Historical Price Trend:** A 30-day historical context of price movements that helps the user identify whether the current price action is an anomaly or part of a larger trend.
- **Future Price Predictions:** It uses a Linear Regression Agent to forecast the price movement for the subsequent five days; in this case, it will provide a time-series graph of the forecast in a very neat format.
- **Market Risk Indicator:** A graphical measure of the Composite Risk Score, produced by both numerical volatility and news sentiment.



The system has its interface fueled by a deep backend structure that allows the data from raw intake to final visualization in a specific pipeline:

- **Stage 1: Data Acquisition:** This stage sees agents 1.1-1.3 scrape news headlines, current prices, and time-series data.
- **Stage 2: Data Processing & Analysis:** Data Processing & Analysis: The Decision Node functions as a very crucial logic gateway. Whenever the Risk Score crosses the threshold level of 5.0, the High-Risk Alert gets automatically generated before the final dashboard display.
- **Stage 3: Prediction System Workflow:** Prediction System Workflow: Final-stage agents (3.1 - 3.3) are involved in synthesizing predictions, having the capability to produce the PNG visual artifacts (charts, tables) for display in the dashboard.

SECTION 5 — Conclusion and References

➤ Conclusion

Problem Solved The market of cryptocurrency is also marked by high volatility and too much data. For traders or analysts, analyzing live market data, extracting news headlines around the world, and determining risks manually is a time-consuming and error-prone process. This problem is resolved by applying a fully automated multi-agent model to perform the end-to-end intelligence process.

System Impact and Main Findings The implementation of this automated system has transformed the data-to-decision lifecycle in the following ways:

- **Efficiency Gains:** The MAS was able to automate entirely the tasks involved in conducting market research every day, cutting down the time it took to compile a thorough report from around 45 minutes of human efforts to under 10 seconds of processing time.
- **Operational Consistency:** The use of the Risk Scoring Agent (Agent 2.4) eliminates the need for human subjective verification and gives the “Sell/Avoid” signal that is objective and consistent, even in highly noisy market conditions.
- **Predictive Clarity:** The fusion of model results within the Linear Regression Agent offered clarity on trends for 5 days, although less complex, which helped to improve productivity by 40-60% when viewing trends on unstructured charts.
- **Synthesized Intelligence:** The utilization of the BART-Large-CNN model confirmed that complex finance variables can indeed be synthesized proficiently into natural language form.

Conclusion In summary, what has been developed here proves that what one could refer to as a “squad” of highly specialized agents, each tackling only one and only one task, whether it is web scraping, sentiment auditing, or forecasting, is more reliable and more robust than one single model.

➤ References

- Beautiful Soup. (2024). *Beautiful Soup 4.12.0 Documentation* [Online]. Available at: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> (Accessed: 15 May 2025).
- CoinDesk. (2025). *Latest Crypto News* [Online]. Available at: <https://www.coindesk.com/> (Accessed: 15 May 2025).
- CoinMarketCap. (2025). *Cryptocurrency Prices, Charts and Market Capitalizations* [Online]. Available at: <https://coinmarketcap.com/> (Accessed: 15 May 2025).
- Hunter, J. D. (2007). 'Matplotlib: A 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90–95.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V. and Zettlemoyer, L. (2020). 'BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension', *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 7871–7880.

- McKinney, W. (2010). 'Data structures for statistical computing in Python', *Proceedings of the 9th Python in Science Conference*, pp. 56–61.
- n8n. (2025). *n8n Documentation: Multi-agent Orchestration* [Online]. Available at: <https://docs.n8n.io/> (Accessed: 12 May 2025).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011). 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12(Oct), pp. 2825–2830.
- Requests. (2024). *Requests: HTTP for Humans™ v2.31.0 Documentation* [Online]. Available at: <https://requests.readthedocs.io/> (Accessed: 14 May 2025).
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M. and Davison, J. (2020). 'Transformers: State-of-the-art natural language processing', *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45.